

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа.

Тема: «Быстрые сортировки»

По дисциплине «Основы алгоритмизации и программирования»

Выполнила: студентка группы РИС-25-26

Абрамова Валерия Андреевна

Принял: доц. Полякова О.А.

Пермь, 2026

Сортировка Ломута

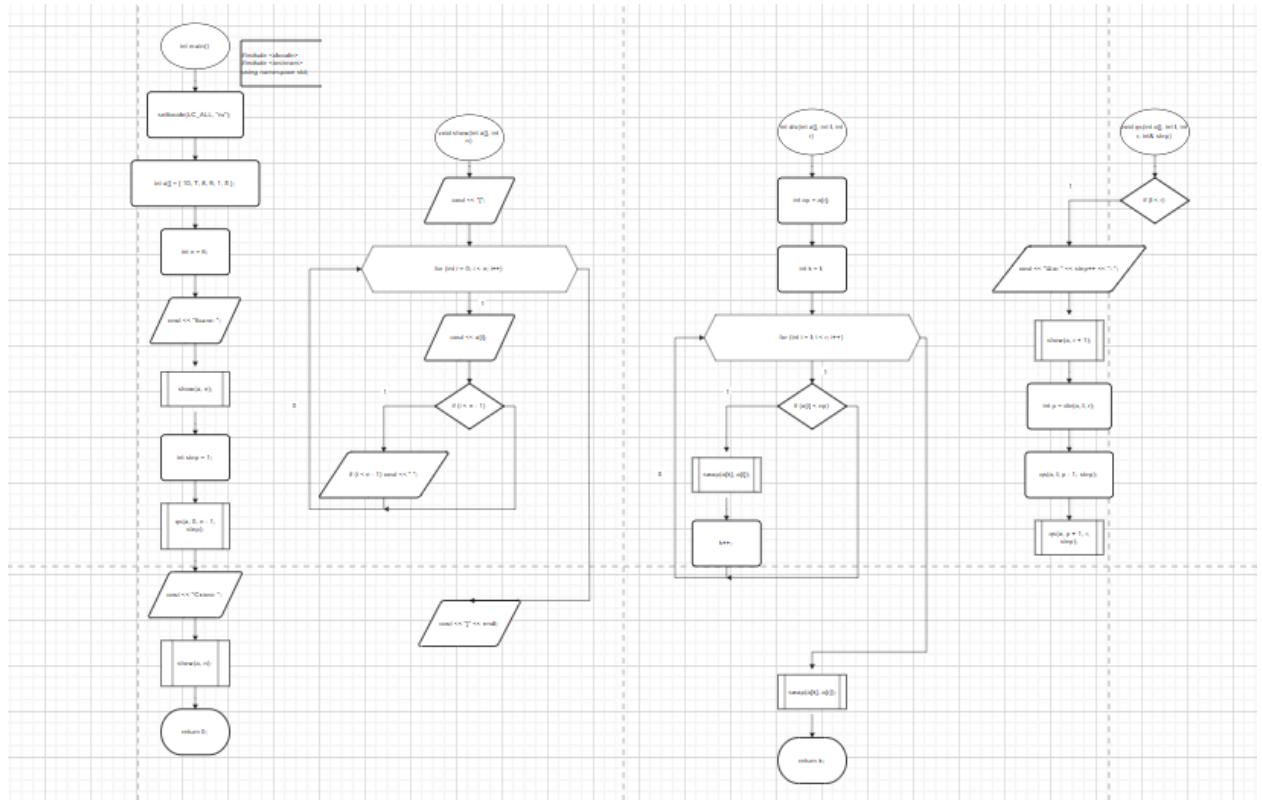
Постановка задачи

Отсортировать массив методом Ломута

Анализ

1. Выбирается опорный элемент – pivot
2. Все элементы меньше опорного располагаем слева от него, все что больше справа
3. Рекурсивно повторяем эти действия, пока не получим отсортированный массив

Блок-схема



Код

```
#include <locale>
#include <iostream>
#include <string>

using namespace std;

void show(int a[], int n, int op = -1, int l = -1, int r = -1) {
    cout << "[";
    for (int i = 0; i < n; i++) {
        if (i == l) cout << "[";
        if (i == op) cout << "(" << a[i] << ")";
        else cout << a[i];
        if (i == r) cout << "]";
        if (i < n - 1) cout << " ";
    }
    cout << "]" << endl;
}

int div(int a[], int l, int r) {
    int op = a[r];
    int k = l;
```

```

cout << " опора=" << op << ", сравниваем: ";
for (int i = l; i < r; i++) cout << a[i] << " ";
cout << endl;

for (int i = l; i < r; i++) {
    if (a[i] < op) {
        swap(a[k], a[i]);
        k++;
    }
}
swap(a[k], a[r]);
return k;
}

void qs(int a[], int l, int r, int& step) {
    if (l < r) {
        cout << "\nшаг " << step++ << " (массив "; show(a, r + 1, r, l, r);

        int p = div(a, l, r);

        cout << " после: "; show(a, r + 1, p, l, r);
        cout << " опора " << a[p] << " на месте" << endl;

        qs(a, l, p - 1, step);
        qs(a, p + 1, r, step);
    }
    else if (l == r) {
        cout << "\nшаг " << step++ << " (1 элемент "; show(a, r + 1, l, l, r);
        cout << " пропуск" << endl;
    }
}

int main() {
    setlocale(LC_ALL, "ru");
    int a[] = { 10, 7, 8, 9, 1, 5 };
    int n = 6;
    cout << "было: "; show(a, n);
    int step = 1;
    qs(a, 0, n - 1, step);
    cout << "стало: "; show(a, n);

    return 0;
}

```

Вывод программы

```
было: [10 7 8 9 1 5]

шаг 1 (массив [[10 7 8 9 1 (5)]]
  опора=5, сравниваем: 10 7 8 9 1
  после: [[1 (5) 8 9 10 7]]
  опора 5 на месте

шаг 2 (1 элемент [[(1)]]
  пропуск

шаг 3 (массив [1 5 [8 9 10 (7)]]
  опора=7, сравниваем: 8 9 10
  после: [1 5 [(7) 9 10 8]]
  опора 7 на месте

шаг 4 (массив [1 5 7 [9 10 (8)]]
  опора=8, сравниваем: 9 10
  после: [1 5 7 [(8) 10 9]]
  опора 8 на месте

шаг 5 (массив [1 5 7 8 [10 (9)]]
  опора=9, сравниваем: 10
  после: [1 5 7 8 [(9) 10]]
  опора 9 на месте

шаг 6 (1 элемент [1 5 7 8 9 [(10)]]
  пропуск
стало: [1 5 7 8 9 10]
```

Сортировка Хоара

Постановка задачи

Отсортировать массив методом Хоара

Анализ задачи

1. Если диапазон менее 2-х элементов, прекращаем действие (массив уже отсортирован)
2. За опорные элемент выбираем элемент в середине
3. Разбиваем диапазон на части: меняем порядок таким образом, чтобы элементы меньше опорного элемента шли до него, а элементы больше — после него
4. Рекурсивно применяем алгоритм к полученным частям

Код

```
#include <iostream>
#include <locale>
using namespace std;
```

```

void printNum(int n) {
    if (n < 0) {
        putchar('-');
        n = -n;
    }
    if (n / 10) printNum(n / 10);
    putchar(n % 10 + '0');
}

void printArray(int arr[], int size) {
    putchar('[');
    for (int i = 0; i < size; i++) {
        printNum(arr[i]);
        if (i < size - 1) {
            putchar(' ');
        }
    }
    putchar(']');
    putchar('\n');
}

void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void quickSort(int arr[], int left, int right, int size) {
    if (left >= right) return;

    int pivot = arr[(left + right) / 2];
    int i = left;
    int j = right;

    cout << "Сортируем часть с " << left << " по " << right;
    cout << ", опорный элемент: " << pivot << endl;

    while (i <= j) {
        while (arr[i] < pivot) i++;
        while (arr[j] > pivot) j--;

        if (i <= j) {
            cout << " Меняем arr[" << i << "]= " << arr[i];
            cout << " и arr[" << j << "]= " << arr[j] << endl;
            swap(&arr[i], &arr[j]);
            cout << " Результат: ";
            printArray(arr, size);
            i++;
            j--;
        }
    }

    cout << "Разделили. Левая часть до " << j << ", правая с " << i << endl;
    cout << "Текущий массив: ";
    printArray(arr, size);
    cout << endl;

    if (left < j) quickSort(arr, left, j, size);
    if (i < right) quickSort(arr, i, right, size);
}

int main() {
    setlocale(LC_ALL, "Russian");

    int arr[] = { 64, 34, 25, 12, 22, 11, 90 };
    int size = sizeof(arr) / sizeof(arr[0]);

    cout << "Исходный массив: ";

```

```

printArray(arr, size);
cout << endl;

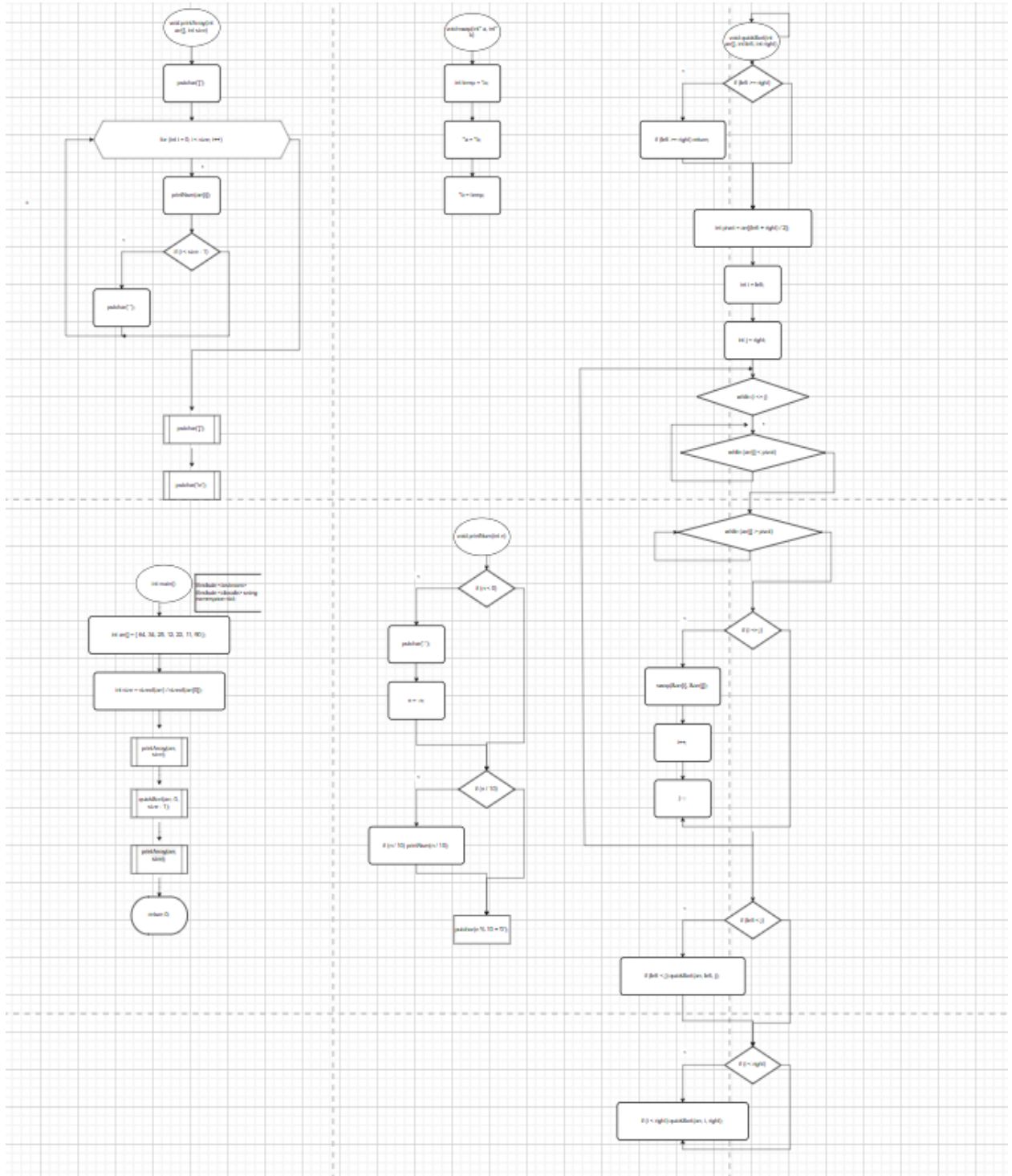
quickSort(arr, 0, size - 1, size);

cout << "Отсортированный массив: ";
printArray(arr, size);

return 0;
}

```

Блок-схема



Вывод программы

```
Исходный массив: [64 34 25 12 22 11 90]

Сортируем часть с 0 по 6, опорный элемент: 12
  Меняем arr[0]=64 и arr[5]=11
  Результат: [11 34 25 12 22 64 90]
  Меняем arr[1]=34 и arr[3]=12
  Результат: [11 12 25 34 22 64 90]
Разделили. Левая часть до 1, правая с 2
Текущий массив: [11 12 25 34 22 64 90]

Сортируем часть с 0 по 1, опорный элемент: 11
  Меняем arr[0]=11 и arr[0]=11
  Результат: [11 12 25 34 22 64 90]
Разделили. Левая часть до -1, правая с 1
Текущий массив: [11 12 25 34 22 64 90]

Сортируем часть с 2 по 6, опорный элемент: 22
  Меняем arr[2]=25 и arr[4]=22
  Результат: [11 12 22 34 25 64 90]
Разделили. Левая часть до 2, правая с 3
Текущий массив: [11 12 22 34 25 64 90]

Сортируем часть с 3 по 6, опорный элемент: 25
  Меняем arr[3]=34 и arr[4]=25
  Результат: [11 12 22 25 34 64 90]
Разделили. Левая часть до 3, правая с 4
Текущий массив: [11 12 22 25 34 64 90]

Сортируем часть с 4 по 6, опорный элемент: 64
  Меняем arr[5]=64 и arr[5]=64
  Результат: [11 12 22 25 34 64 90]
Разделили. Левая часть до 4, правая с 6
Текущий массив: [11 12 22 25 34 64 90]

Отсортированный массив: [11 12 22 25 34 64 90]

C:\Users\Валерия\source\repos\ConsoleApplication32\x64\Debug\ConsoleApplicat
Нажмите любую клавишу, чтобы закрыть это окно:
```

Сортировка Шелла

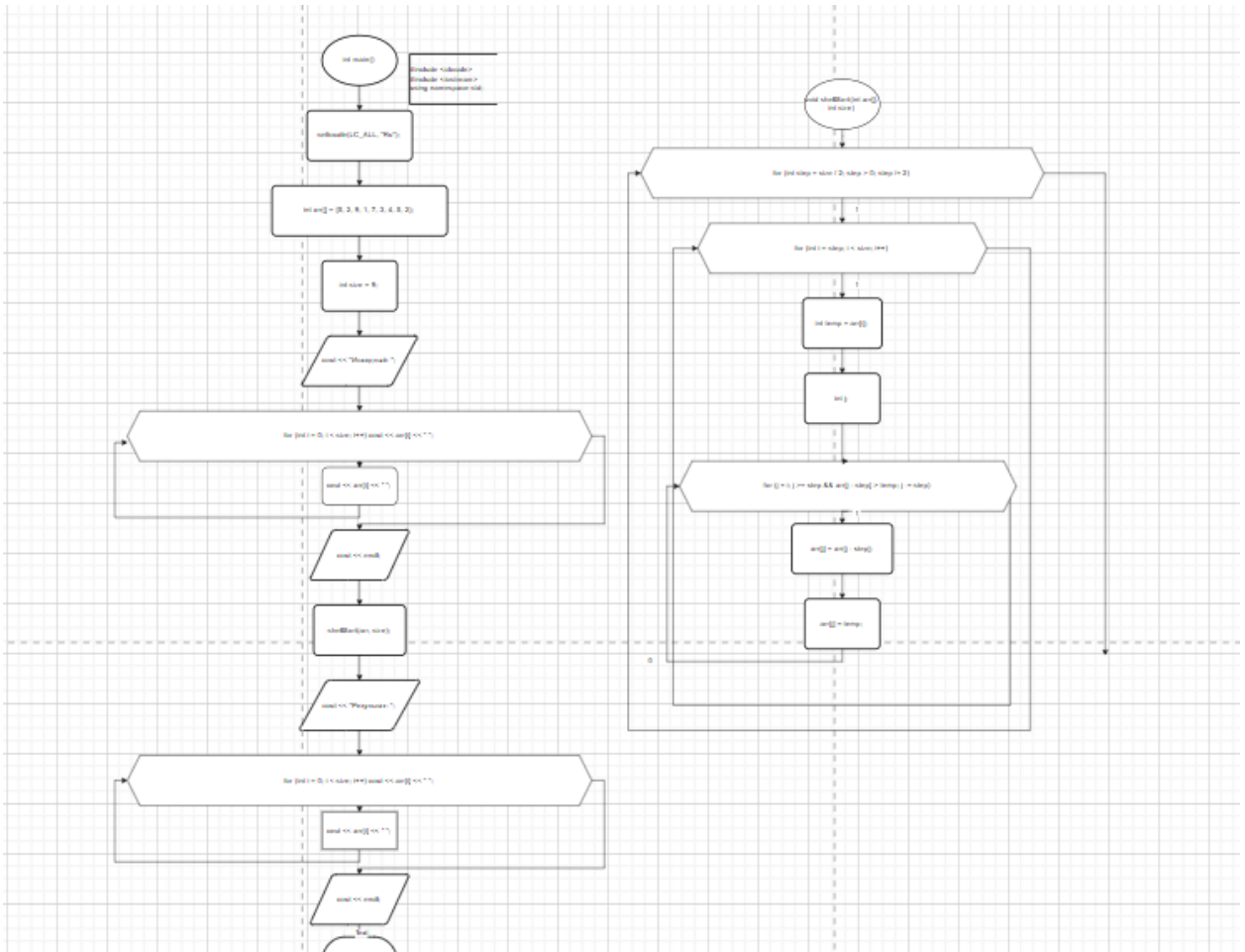
Постановка задачи

Отсортировать массив методом Шелла

Анализ задачи

1. Вычисляется некоторое расстояние между элементам, которое называется шаг
2. Массив разбивается на списки, которые отстают друг от друга на шаг
3. Каждый из списков сортируется методом вставки
4. Возвращаемся к шагу 2, когда полученные списки объединились и уменьшается шаг. алгоритм повторяется до тех пор, пока шаг > 0

Блок-схема



Код

```

#include <locale>
#include <iostream>
using namespace std;

void shellSort(int arr[], int size) {
    for (int step = size / 2; step > 0; step /= 2) {
        cout << "\nШаг с gap = " << step << endl;

        for (int i = step; i < size; i++) {
            int temp = arr[i];
            int j;

            cout << " Вставляем " << temp << " с позиции " << i << endl;

            for (j = i; j >= step && arr[j - step] > temp; j -= step) {
                cout << " " << arr[j - step] << " > " << temp << " -> сдвигаем "
                    << arr[j - step] << " на позицию " << j << endl;
                arr[j] = arr[j - step];
            }

            arr[j] = temp;
            cout << " Ставим " << temp << " на позицию " << j << endl;
            cout << " Массив: ";
            for (int k = 0; k < size; k++) cout << arr[k] << " ";
            cout << endl;
        }
    }
}

```



```

    }
}

int main() {
    setlocale(LC_ALL, "Ru");
    int arr[] = { 5, 2, 9, 1, 7, 3, 4, 5, 2 };
    int size = 9;

    cout << "Исходный: ";
    for (int i = 0; i < size; i++) cout << arr[i] << " ";
    cout << endl << endl;

    shellSort(arr, size);

    cout << "\nРезультат: ";
    for (int i = 0; i < size; i++) cout << arr[i] << " ";
    cout << endl;

    return 0;
}

```

Результат выполнения программы

Исходный: 5 2 9 1 7 3 4 5 2

Шаг с гар = 4

Вставляем 7 с позиции 4
Ставим 7 на позицию 4
Массив: 5 2 9 1 7 3 4 5 2
Вставляем 3 с позиции 5
Ставим 3 на позицию 5
Массив: 5 2 9 1 7 3 4 5 2
Вставляем 4 с позиции 6
9 > 4 -> сдвигаем 9 на позицию 6
Массив: 5 2 9 1 7 3 9 5 2
Ставим 4 на позицию 2
Массив: 5 2 4 1 7 3 9 5 2
Вставляем 5 с позиции 7
Ставим 5 на позицию 7
Массив: 5 2 4 1 7 3 9 5 2
Вставляем 2 с позиции 8
7 > 2 -> сдвигаем 7 на позицию 8
Массив: 5 2 4 1 7 3 9 5 7
5 > 2 -> сдвигаем 5 на позицию 4
Массив: 5 2 4 1 5 3 9 5 7
Ставим 2 на позицию 0
Массив: 2 2 4 1 5 3 9 5 7

Шаг с гар = 2

Вставляем 4 с позиции 2
Ставим 4 на позицию 2
Массив: 2 2 4 1 5 3 9 5 7
Вставляем 1 с позиции 3
2 > 1 -> сдвигаем 2 на позицию 3
Массив: 2 2 4 2 5 3 9 5 7
Ставим 1 на позицию 1
Массив: 2 1 4 2 5 3 9 5 7
Вставляем 5 с позиции 4
Ставим 5 на позицию 4
Массив: 2 1 4 2 5 3 9 5 7
Вставляем 3 с позиции 5
Ставим 3 на позицию 5
Массив: 2 1 4 2 5 3 9 5 7
Вставляем 9 с позиции 6
Ставим 9 на позицию 6
Массив: 2 1 4 2 5 3 9 5 7
Вставляем 5 с позиции 7
Ставим 5 на позицию 7
Массив: 2 1 4 2 5 3 9 5 7
Вставляем 7 с позиции 8
9 > 7 -> сдвигаем 9 на позицию 8
Массив: 2 1 4 2 5 3 9 5 9
Ставим 7 на позицию 6
Массив: 2 1 4 2 5 3 7 5 9

Шаг с гар = 1

Вставляем 1 с позиции 1
2 > 1 -> сдвигаем 2 на позицию 1

Вставляем 3 с позиции 5
Ставим 3 на позицию 5
Массив: 2 1 4 2 5 3 9 5 7
Вставляем 9 с позиции 6
Ставим 9 на позицию 6
Массив: 2 1 4 2 5 3 9 5 7
Вставляем 5 с позиции 7
Ставим 5 на позицию 7
Массив: 2 1 4 2 5 3 9 5 7
Вставляем 7 с позиции 8
9 > 7 -> сдвигаем 9 на позицию 8
Массив: 2 1 4 2 5 3 9 5 9
Ставим 7 на позицию 6
Массив: 2 1 4 2 5 3 7 5 9

Шаг с гар = 1

Вставляем 1 с позиции 1
2 > 1 -> сдвигаем 2 на позицию 1
Массив: 2 2 4 2 5 3 7 5 9
Ставим 1 на позицию 0
Массив: 1 2 4 2 5 3 7 5 9
Вставляем 4 с позиции 2
Ставим 4 на позицию 2
Массив: 1 2 4 2 5 3 7 5 9
Вставляем 2 с позиции 3
4 > 2 -> сдвигаем 4 на позицию 3
Массив: 1 2 4 4 5 3 7 5 9
Ставим 2 на позицию 2
Массив: 1 2 2 4 5 3 7 5 9
Вставляем 5 с позиции 4
Ставим 5 на позицию 4
Массив: 1 2 2 4 5 3 7 5 9
Вставляем 3 с позиции 5
5 > 3 -> сдвигаем 5 на позицию 5
Массив: 1 2 2 4 5 5 7 5 9
4 > 3 -> сдвигаем 4 на позицию 4
Массив: 1 2 2 4 4 5 7 5 9
Ставим 3 на позицию 3
Массив: 1 2 2 3 4 5 7 5 9
Вставляем 7 с позиции 6
Ставим 7 на позицию 6
Массив: 1 2 2 3 4 5 7 5 9
Вставляем 5 с позиции 7
7 > 5 -> сдвигаем 7 на позицию 7
Массив: 1 2 2 3 4 5 7 7 9
Ставим 5 на позицию 6
Массив: 1 2 2 3 4 5 5 7 9
Вставляем 9 с позиции 8
Ставим 9 на позицию 8
Массив: 1 2 2 3 4 5 5 7 9

Результат: 1 2 2 3 4 5 5 7 9